Drishti

Study notes and viva questions with explained answers

Paper: Drishti: Temporal Fusion of Eye Closure and Yawning for Driver Monitoring.

Purpose: Understand the project, explain the implementation, defend the results, and answer difficult questions without overstating the work.

Basis: The current illustrated paper, temporal_engine.py, the verification script and recorded results, and the scripts that generate the figures. IVP is interpreted here as Image and Video Processing. These notes describe the current checkout, not an assumed complete application.

Read this first	What it means
Implemented	A rule-based temporal engine accepts EAR and MAR, filters them, tracks events, calculates evidence, and returns four warning states.
Demonstrated	Eight controlled synthetic scenarios passed their stated assertions. Additional synthetic runs show closure/recovery behavior and sampling-rate effects.
Not yet established	Real-driver accuracy, robustness to talking and looking away, physiological drowsiness validity, and end-to-end camera-to-warning latency.
Safe central claim	We verified selected software behaviors of a temporal decision component; an integrated real-video evaluation remains necessary.

How to use these notes

Read Parts 1-7 to understand the concepts and implementation. Use Part 8 as a section-by-section explanation of the paper. Practise the numbered questions in Part 9 aloud. Finish with the short project explanation and checklist in Part 10.

Keep three categories separate in every answer: what the code implements, what the tests actually checked, and what you propose as future work. A diagram of an intended camera pipeline does not prove that every block is implemented.

1. Problem, solution and scope

The problem

One image cannot fully explain a driver's condition. A closed eye may be an ordinary blink or a deliberate closure. An open mouth may be speech, singing, breathing, or a yawn. Measurements may also fluctuate because of tracking errors. The system therefore needs duration and recent context, rather than a decision based only on one observation.

The proposed solution

Drishti combines eye and mouth measurements across time. It filters the measurements, maintains eye and yawn states, times events, remembers recent events, and combines evidence into warning rules. Temporal means related to time; fusion means combining evidence; an engine is the component that performs the processing.

```
Intended: camera -> landmarks -> EAR/MAR -> temporal engine -> warning
```

```
Verified component: supplied EAR/MAR -> temporal engine -> score/status
```

The current checkout contains the temporal engine but no camera-capture or facial-landmark front end. A comment mentions main.py, but that file is absent. Do not claim the current tests demonstrate a complete camera application.

The output states

ALERT, FATIGUED, DROWSY and CRITICAL are operational labels selected by rules. They have not been validated as clinical or physiological diagnoses. ALERT means no higher rule applies; it is not a guarantee that the person is fully attentive.

2. Features and IVP concepts

Frames, landmarks and features

A frame is one image in a video. A landmark is a point on a facial structure, such as an eye corner or eyelid. A feature is a numerical description derived from an observation. A scalar feature is one number. The engine accepts two scalar features: eye aspect ratio (EAR) and mouth aspect ratio (MAR).

EAR: eye aspect ratio

```
EAR = (distance(p2,p6) + distance(p3,p5)) / (2 * distance(p1,p4))
```

Six points describe the eye. The numerator contains two vertical distances; the denominator contains twice the horizontal width. As the eye closes, the vertical distances usually decrease, so EAR decreases. The double bars in the paper mean Euclidean distance between points.

Worked example: If both vertical distances are 6 pixels and the width is 30 pixels, EAR = (6 + 6) / (2 x 30) = 0.20. If the vertical distances become 10 pixels, EAR is about 0.333. A ratio reduces sensitivity to uniform image scaling, but not to all pose, tracking or anatomical differences.

Scope: The engine receives EAR; it does not detect landmarks or calculate these distances. The front end must define landmark indices, normalization, and whether both eyes are averaged.

MAR: mouth aspect ratio

MAR expresses mouth opening relative to a mouth dimension. Under the expected convention, a larger value indicates a more open mouth. The exact geometry is external to the engine. Different MAR formulas can yield different ranges, so a threshold is meaningful only with the corresponding definition.

IVP concept	How it relates to this project
Feature extraction	A proposed front end converts images to EAR and MAR; the tested engine receives the resulting numbers.
Temporal median filtering	Implemented on successive numerical EAR/MAR observations, not neighboring image pixels.
Thresholding	Implemented to classify eye state and identify possible mouth-opening events.
Temporal analysis	Implemented through timers, state memory and rolling event histories.
Feature fusion	Implemented through score contributions and combined warning conditions.
CNN, LSTM, SVM	Discussed in related work; not trained or implemented in the temporal engine.

3. Filtering and hysteresis

Median filtering: stabilize the numbers

The engine stores up to seven recent EAR values and seven recent MAR values in deque(maxlen=7). Adding an eighth value removes the oldest. NumPy calculates the median separately for each history. During startup it uses the observations already available; it does not wait for seven.

```
raw: 0.34, 0.35, 0.33, 0.05, 0.34, 0.36, 0.35 -> median: 0.34
```

The isolated low reading does not dominate the output. If four of seven values become low, the median can follow the sustained change. With an even number of startup observations, the median is the average of the two middle values.

Trade-off: The filter rejects isolated fluctuations but delays genuine transitions and can suppress very short movements. Seven samples at 20 Hz span six 0.05-second intervals, or 0.30 seconds. In the reported persistent-step experiment, the filtered state changed 0.15 seconds after the raw step.

Hysteresis: stabilize the state

Hysteresis uses different entry and exit thresholds. The state depends partly on what it was previously. OPEN changes to CLOSED at filtered EAR <= 0.23. CLOSED changes to OPEN at EAR >= 0.27. Between those values the previous state remains unchanged.

Filtered EAR	Previous state	Result
0.30	OPEN	OPEN
0.25	OPEN	OPEN
0.22	OPEN	CLOSED
0.25	CLOSED	CLOSED
0.28	CLOSED	OPEN

With a single cutoff at 0.25, values 0.24, 0.26, 0.24, 0.26 could repeatedly switch state. Two thresholds prevent this chattering. The familiar analogy is a thermostat with separate on and off temperatures.

Difference: Filtering changes the numerical feature using recent samples. Hysteresis uses the current filtered feature and previous state to decide whether to switch. Neither identifies the cause of eye closure or mouth opening.

4. Events, durations and memory

Eye closure and blinks

On the first CLOSED update, the engine records eye_closed_start. Current closure duration is now minus that timestamp. On reopening, it evaluates the completed closure. A duration from 0.08 through 0.65 seconds counts as a blink. A continuing closure reaching 0.8 seconds registers a long-closure event.

current_closure_registered prevents counting a single continuous long closure many times. It resets when the eyes reopen. Durations greater than 0.65 but below 0.8 seconds satisfy neither event category. Timings refer to the filtered eye state, which may differ from physical eyelid motion.

Yawn candidate, confirmation and reset

Filtered MAR >= 0.42 starts a candidate. The condition must continue for at least 0.65 seconds to confirm a yawn. Confirmation increments the cumulative counter and stores one timestamp. The active yawn ends when MAR <= 0.28. Before confirmation, dropping below 0.42 cancels the candidate. Another yawn requires the previous active event to end.

A candidate is only a possible event. The timer rejects some brief openings; it does not prove the movement was a physiological yawn. Talking or open-mouth breathing that satisfies the rule can still be counted.

History / counter	Meaning
Recent yawns: 45 seconds	Confirmed yawn timestamps within the memory window.
Recent long closures: 30 seconds	Long-closure registration timestamps within the window.
Closed-eye samples: 30 seconds	Timestamped binary eye states used for the sample ratio.
Cumulative count	Total counted events since this engine instance began; does not expire.
Recent count	Only events still retained after cleanup.

Cleanup removes an event when its age is greater than its window. An event exactly at the boundary remains until a later update. Histories count registrations, not the fraction of time occupied by an event. An unusually long active yawn can outlast its retained registration timestamp.

5. PERCLOS and the evidence score

Three different questions

Quantity	Question it answers
Current closure duration	How long have the eyes been continuously classified CLOSED this time?
Closed-eye sample ratio	What fraction of recent retained observations were classified CLOSED?
Evidence score	How much combined warning evidence is present under our selected rules?

PERCLOS versus the implemented approximation

Physiological PERCLOS concerns the proportion of time that eyelids are at least 80% closed over an observation interval. The implementation does not measure 80% eyelid coverage. It labels each observation OPEN or CLOSED using EAR and calculates a sample fraction. The paper therefore calls it a PERCLOS-inspired sample proxy.

```
Psample = number of retained CLOSED observations / all retained observations
```

Example: If 40 of 100 retained observations are CLOSED, Psample = 0.40. At regular sampling, it approximates 40% of the observed time. At irregular sampling, equal observation weights do not represent equal durations. The ratio includes short blinks and uses incomplete history during startup.

Startup: One CLOSED observation gives 1 / 1 = 1. This does not mean a complete 30-second period was spent closed. The cold-start test returns ratio 1, current duration 0 seconds, and ALERT. A high ratio alone does not automatically trigger DROWSY.

What does the evidence score do?

The target score adds points for current closure, the sample ratio, recent yawns, an active yawn, recent long closures, and a combined-cue bonus. It compresses those inputs into a bounded engineering score from 0 to 100. The displayed score smooths that target. Warning rules use the rounded displayed score alongside direct duration and history conditions.

A score of 80 is 80 evidence points, not an 80% probability of sleepiness. The weights were hand-set, not learned or calibrated as probabilities on labeled drivers.

```
T = min(100, C(duration) + E(ratio) + W(yawns) + A(active) + H(long closures) + bonus)
```

Contribution	Thresholds and points
C: current closure	At least 0.8 / 1.5 / 2.0 / 3.0 seconds -> 20 / 45 / 65 / 85 points.
E: sample ratio	At least 0.20 / 0.30 / 0.40 / 0.50 -> 10 / 18 / 25 / 30 points.
W: recent yawns	1 / 2 / at least 3 -> 10 / 20 / 30 points.
A: active yawn	Confirmed yawn currently active -> 5 points.
H: recent long closures	1 / 2-3 / at least 4 -> 8 / 18 / 25 points.
Bonus	10 points if duration >= 0.8 seconds and either recent yawns >= 2 or ratio >= 0.35. Added once.

Each contribution uses only its highest satisfied interval. A three-second closure contributes 85 points, not 20 + 45 + 65 + 85. Contributions are zero below their first threshold.

Worked score example

Suppose duration = 1.0 second, ratio = 0.40, recent yawns = 2, no active yawn, and one recent long closure. The target is 20 + 25 + 20 + 0 + 8 + 10 = **83**. The displayed score may still be lower because it approaches the target gradually.

Score smoothing and recovery

```
Snew = Sprevious + alpha * (T - Sprevious)
```

Alpha is 0.20 while the target exceeds the previous score and 0.08 otherwise. A rise from S = 20 toward T = 80 gives 32 after one update. A fall from S = 80 toward T = 20 gives 75.2 before extra recovery. OPEN eyes with no active yawn cause another 0.12-point subtraction per update. The score is clipped to 0-100 and rounded for output.

The intention is fast escalation and slower recovery. The trade-off is persistence after reopening and dependence on update frequency. This behavior has not been validated as the appropriate warning recovery time for real drivers.

Warning priority: check the most severe rule first

State	Conditions, evaluated in this order
CRITICAL	Duration >= 3.0 s; OR duration >= 2.0 s with at least 2 recent yawns.
DROWSY	Otherwise: duration >= 1.5 s; OR ratio >= 0.45 with at least 2 recent long closures; OR displayed score >= 55 with (ratio >= 0.35 OR duration >= 0.8 s).
FATIGUED	Otherwise: at least 2 recent yawns OR displayed score >= 30.
ALERT	None of the higher-priority rules applies.

There is no simple rule that every score above one threshold means CRITICAL. Direct severe-closure rules can trigger regardless of the displayed score. Conversely, the score-based DROWSY branch requires supporting eye evidence.

6. Files and implementation flow

Paths below are relative to the project root. Document-generation scripts create outputs; they are not detection modules.

File / location	What it does
temporal_engine.py	Main implementation. TemporalEngine stores state; update(raw_ear, raw_mar) filters features, tracks events, scores evidence and returns results.
research_paper/support/verify_engine.py	Supplies synthetic inputs and a patched clock, runs assertions, and writes verification results.
research_paper/support/verification_results.json	Records update rate, scenario outcomes and a SHA-256 fingerprint of the tested source.
research_paper/rewrite_build/make_ieee_figures.py	Creates diagrams and plots and runs synthetic sequences for score and update-rate comparisons.
closure_trace_data.json; rate_sweep_data.json	Stored numerical outputs from controlled figure-generation runs in rewrite_build/.
research_paper/rewrite_build/build_ieee_visual.py	Builds the current illustrated Word paper: text, two-column body, tables, images, captions and references.
support/build_paper.py; rewrite_build/rewrite_paper.py	Earlier paper-generation scripts. They do not perform runtime detection.
research_paper/support/add_technical_figures.py	Supporting technical figure-generation work.
research_paper/presentation_build/content_and_speech.py	Prepares presentation content and creates the speech PDF.
research_paper/presentation_build/build_deck.mjs	JavaScript module that builds the PowerPoint deck.
research_paper/assets/; research_paper/output/	Visual assets and generated research/presentation deliverables.
venv/; __pycache__/; .git/	Python environment, automatic bytecode cache, and version-control data. These are not algorithm stages.

The three main methods

__init__(): Creates thresholds, histories, counters, initial OPEN state, yawn variables and zero score. **_clean_old_events(now):** Expires samples and event timestamps. **update(raw_ear, raw_mar):** Performs one processing step and returns a dictionary.

```
engine = TemporalEngine()
result = engine.update(raw_ear, raw_mar)
```

Create one engine for a stream and reuse it. Creating a fresh engine for every frame would reset all memory and timers. A fresh engine is useful between independent test scenarios or separate sessions.

Libraries: time provides timestamps; NumPy supplies medians; collections.deque stores histories. unittest.mock.patch is used by test scripts to control time. The engine returns filtered features, eye state, closure duration, counters, recent histories, ratio, target score, displayed score and status.

7. How the evaluation was done and what it showed

Controlled synthetic verification

The test script supplies numbers rather than recorded images. Typical inputs are open EAR 0.35, closed EAR 0.15, neutral MAR 0.15 and mouth-opening MAR 0.60. A patched clock advances 0.05 seconds per update, representing 20 Hz. Hz means updates per second; it is not a measured camera speed in this experiment.

Each scenario starts with a fresh engine. Assertions check expected states and counters. SHA-256 records a fingerprint of the implementation, helping identify which version was tested. A hash does not prove that the code is correct.

Scenario	What the assertions checked
Open-eye baseline	ALERT, zero score and zero closed-eye ratio.
Eye hysteresis	CLOSED persists through intermediate EAR; sufficiently high EAR reopens it.
Short closure	One blink and no long-closure event.
Sustained closure	DROWSY then CRITICAL, with one long-closure registration.
Brief mouth opening	No confirmed yawn.
Two yawns	Two separately confirmed events and FATIGUED.
Yawn memory expiry	Recent count returns to zero; cumulative count persists.
Closed-eye startup	Ratio 1, duration 0 seconds, and ALERT.

All eight passed their stated assertions. This is functional verification, not 100% recognition accuracy. It does not cover every boundary, interaction, missing input or real behavior.

Closure and recovery trace at 20 Hz

Time	Observation
1.05 s	Raw EAR changes to the closed-eye value.
1.20 s	Filtered state becomes CLOSED: 0.15 s after the raw step.
2.10 s	FATIGUED appears; displayed score 34.
2.35 s	DROWSY appears at about 1.15 s filtered closure; score 55.
4.20 s	CRITICAL appears when filtered closure reaches 3.0 s.
5.20 s	Filtered eyes reopen; DROWSY remains with score 95.
5.95 s	State falls to FATIGUED with score 53.

The score-and-ratio branch can trigger before the direct 1.5-second duration rule. Historical samples and smoothing explain warning persistence after reopening. These are observed software interactions, not proof of appropriate real-driver warning timing.

Sampling-rate comparison

Rate	Delay to filtered CLOSED	Delay to first DROWSY
10 Hz	0.30 s	1.80 s
20 Hz	0.15 s	1.30 s
30 Hz	0.10 s	About 1.17 s

This separate experiment sets raw closure at 1.00 seconds so it aligns with all sampling grids. Its timing differs slightly from the preceding trace. Seven-sample filtering and per-update score coefficients both change their effect in seconds with rate. The experiment does not isolate their individual contributions or identify an optimal frame rate.

8. Explain each paper section and visual

Paper part	Purpose and how to explain it
Title and abstract	Name the temporal-fusion problem, summarize the rule-based method, controlled tests, findings and unmeasured real-world performance.
I. Introduction	Explain why single observations are ambiguous and why duration and recent history are needed. State the component-level research question.
II. Related Work	Connect EAR, PERCLOS, temporal models and yawn datasets to the design. Explain that prior-study metrics are not Drishti results.
III. Inputs and Verification Protocol	Describe synthetic features, 20 Hz updates, patched timestamps, fresh instances and assertions so the test is reproducible.
IV. Method	Explain feature filtering, hysteresis, timers, event history, sample ratio, evidence score and prioritized warning rules.
V. Results	Report the eight passing checks, closure/recovery trace and sampling-rate experiment. Explain exactly what each result establishes.
VI. Discussion and Conclusion	Interpret the interactions, identify limitations, propose calibration and independent real-video evaluation, and give a limited conclusion.
References	Credit sources for concepts, related results, datasets and images. A citation is not evidence that our code was evaluated on that resource.

Visual	What it explains
Figure 1	Educational/resting-posture context. Not project experimental data and not proof of impairment.
Figure 2	Published UTA-RLDD examples, not frames used in the functional checks.
Figures 3-6	Pipeline boundary, EAR geometry, eye hysteresis and yawn state logic.
Figures 7-8	Synthetic closure/recovery response and target versus displayed score.
Figure 9	Synthetic warning-delay comparison across update rates.
Table 1	Literature datasets, methods, reported measurements and gaps. Its other-study accuracy values are not our accuracy.
Table 2	Our eight deterministic verification scenarios and asserted outcomes.

Advantages

The rules are inspectable, selected events are counted once, temporal context distinguishes some short and sustained movements, and multiple cues contribute to warnings. The engine requires no model training and its selected behaviors can be reproduced with controlled inputs. This does not establish that it outperforms trained models.

Limitations and ambiguous behavior

Talking, singing or sustained open-mouth breathing can satisfy the yawn rule. Voluntary eye closure can satisfy the same rules as fatigue-related closure. Looking away can distort EAR, and there is no head-pose or gaze check. Filtering and hysteresis reduce fluctuations; they do not determine the cause of the behavior. No real-video test has established reliability on these cases.

Other limitations include fixed thresholds, unspecified landmark/MAR front end, filtering delay, startup history, sampling-rate dependence, slow recovery, and no explicit invalid-input or missing-observation policy. time.time() reads wall-clock time; a monotonic clock is preferable for elapsed live timing, while recorded video needs its own timestamps. These are proposed improvements, not implemented fixes.

What a stronger evaluation needs

Define the front end and its confidence policy; calibrate on separate subjects; freeze thresholds; test on independently labeled videos including competing behaviors. Report event precision, recall, F1, state confusion matrix and macro-F1, false warnings per hour, missed sustained closures, detection delay and end-to-end latency on named hardware. Compare against simpler baselines and remove individual components in ablations. None of those unmeasured metrics should be invented.

9. Viva questions and explained answers

These are speaking guides. Use the factual answers for the current implementation. For personal contribution questions, describe only work you actually performed and can demonstrate.

A. Project purpose and contribution

Q1. Explain your project in simple words.

Answer: Drishti watches how eye and mouth measurements change over time. It filters the numbers, times closures and mouth openings, remembers events, and applies explicit rules to produce warning states. The tested contribution is the temporal decision engine, not a fully evaluated camera application.

Q2. What exact problem are you solving?

Answer: A single closed-eye or open-mouth observation is ambiguous. We address the temporal decision problem: how duration, repetition and combined cues should affect a warning. Our tests establish selected software behaviors; they do not prove that every ambiguous real behavior is resolved.

Q3. Why is the title called temporal fusion?

Answer: Temporal means using time and sequence history. Fusion means combining evidence from closure duration, eye-state history and yawning. The engine combines those measurements through hand-set score terms and logical warning conditions.

Q4. What is your main contribution or novelty?

Answer: An explicit temporal decision policy and reproducible functional checks of its behavior, including interactions between scoring, duration and recovery. EAR, median filters and hysteresis are established ideas. We should not claim a new physiological measure, a new trained model, or field-wide novelty without a broader comparison.

Q5. Is this an AI or machine-learning model?

Answer: The examined engine is rule-based. It has no training loop or learned parameters. A future landmark front end might use a learned model, but that is separate. Using NumPy and producing intelligent-looking decisions does not by itself make this component a trained model.

Q6. What did you achieve?

Answer: Eight synthetic scenarios satisfied their assertions. Additional runs exposed early score-based DROWSY activation, warning persistence after reopening and update-rate-dependent timing. These are reproducible implementation findings. Real-driver recognition performance remains unmeasured.

Q7. What did you personally do?

Answer: State your actual contribution, with evidence: for example, implementing a function, designing or checking scenarios, analyzing traces, reviewing sources, or preparing diagrams. Do not copy a role you did not perform. A strong answer names one concrete artifact, explains your decision, and shows how you checked it.

Q8. Why should someone use this approach?

Answer: Its decisions can be inspected and its logic is straightforward to test. It is a useful prototype or baseline for a temporal warning policy. Its suitability for deployment must be established by integrated evaluation; no comparison here proves it is the best approach.

B. Features, filtering and states

Q9. What is EAR and why use a ratio?

Answer: EAR compares two vertical eye distances with horizontal eye width. It generally decreases as the eyelids close. A ratio reduces sensitivity to uniform scale changes, but pose and individual anatomy still matter. Our engine receives EAR rather than extracting it from images.

Q10. What is MAR, and is one formula universal?

Answer: MAR represents mouth opening relative to a mouth dimension. Different point selections and formulas produce different ranges. The front end must specify the convention before the engine's 0.42 and 0.28 thresholds can be interpreted or transferred.

Q11. Why not classify directly from raw EAR?

Answer: One unusual measurement could cause a false state change. The median reduces the effect of isolated outliers. It introduces a delay and can suppress short genuine events, so filtering is a trade-off rather than an automatic improvement in accuracy.

Q12. Why median instead of mean?

Answer: An extreme value can shift a mean substantially. The median depends on ordered position and is resistant to isolated extremes. We chose a short temporal median, but did not conduct a real-video comparison proving it superior to every alternative.

Q13. Why seven observations?

Answer: Seven is the selected code parameter. Four new values generally dominate a full seven-sample window after a persistent step. It has not been optimized or validated across users and rates. A proper comparison would evaluate several window lengths under matched conditions.

Q14. What is hysteresis?

Answer: It uses different switching thresholds and remembers the previous state. OPEN becomes CLOSED at EAR <= 0.23; CLOSED becomes OPEN at EAR >= 0.27. Between them the state remains unchanged, reducing chattering near a boundary.

Q15. How are hysteresis and filtering different?

Answer: Filtering stabilizes the measurement using recent samples. Hysteresis stabilizes the classification using the current filtered measurement and previous state. The engine applies both, then measures event duration.

Q16. How do you recognize a blink?

Answer: On reopening, the engine checks the completed filtered closure. A duration from 0.08 through 0.65 seconds increments the blink count. It recognizes a timing-defined closure event, not the psychological reason for that closure.

Q17. How do you avoid counting one long closure many times?

Answer: The engine sets current_closure_registered after recording the long event at 0.8 seconds. Later updates in the same closure cannot register it again. Reopening resets the flag for the next closure.

Q18. How do you confirm a yawn?

Answer: Filtered MAR must remain at least 0.42 for 0.65 seconds. The engine then counts once and holds the event active until MAR is at most 0.28. A candidate dropping below 0.42 before confirmation is cancelled.

Q19. Why is a separate mouth stop threshold needed?

Answer: After confirmation, small variations near the start threshold should not repeatedly end and restart the same event. The lower 0.28 stopping threshold provides hysteresis. It stabilizes the event but does not prove it was a real yawn.

Q20. What is the difference between recent and cumulative counts?

Answer: Cumulative counts retain the session total. Recent counts retain only timestamps within a window, such as 45 seconds for yawns. Expiring a timestamp reduces recent evidence without erasing the session total.

C. PERCLOS, scores and warning decisions

Q21. What is PERCLOS?

Answer: The physiological concept concerns the proportion of time eyelids are at least 80% closed. Our engine does not measure that coverage; it computes the fraction of retained observations classified CLOSED. We therefore call it a PERCLOS-inspired sample proxy.

Q22. Give a sample-ratio example.

Answer: If 40 of 100 retained updates are CLOSED, the ratio is 0.40. At uniform intervals it approximates 40% of the observed time. It is not 40% confidence that the driver is drowsy, and it need not represent a fully populated 30-second window.

Q23. Why does the startup ratio become one?

Answer: With one retained CLOSED observation, the numerator and denominator are both one. The ratio is mathematically correct for available samples but has very little history. The startup test returns ALERT because severe warning rules are not satisfied at zero closure duration.

Q24. What is the evidence score used for?

Answer: It combines several warning-related quantities into a 0-100 engineering score. The smoothed displayed score supports FATIGUED and DROWSY rules, while direct duration rules can also trigger warnings. It makes multiple contributions explicit, but is not a validated risk probability.

Q25. Does a score of 80 mean 80% sleepy?

Answer: No. It means 80 evidence points under the selected weights and smoothing. A probability claim would require a defined outcome, representative labeled data, and calibration and validation. Those steps were not performed.

Q26. Were the score weights learned?

Answer: No. They are hand-set constants. For example, current closure at least three seconds contributes 85 target points. We can explain their intended ordering, but cannot claim that the precise values are empirically optimal.

Q27. What is target score versus displayed score?

Answer: Target score is the current sum of rule contributions, capped at 100. Displayed score gradually approaches it using alpha 0.20 while rising and 0.08 otherwise, plus an extra recovery subtraction under open/no-yawn conditions. The status uses the rounded displayed score where required.

Q28. Why can DROWSY appear before 1.5 seconds?

Answer: The 1.5-second rule is one branch. Another branch requires displayed score >= 55 with ratio >= 0.35 or closure >= 0.8 seconds. In the reported trace that combination triggered at about 1.15 seconds of filtered closure.

Q29. Why does the warning persist after the eyes reopen?

Answer: Current duration resets, but retained closed-eye samples and event history still contribute, and the displayed score decreases gradually. Persistence is intended by the policy; whether the resulting duration is appropriate needs real-world validation.

Q30. Why check CRITICAL first?

Answer: Several warning conditions may hold at once. Ordered checks ensure the most severe applicable state wins. Otherwise a lower-level condition could hide a critical current closure.

Q31. Can a high score alone trigger CRITICAL?

Answer: No. CRITICAL requires current closure >= 3 seconds, or closure >= 2 seconds with at least two recent yawns. The code does not define CRITICAL as simply a score band.

D. Experiments, graphs and evidence

Q32. Which dataset did you train on?

Answer: We did not train this rule-based engine. Functional verification supplied synthetic EAR and MAR values. YawDD, YawDD+ and UTA-RLDD are discussed as related resources and possible future evaluation data, not as a completed training experiment.

Q33. Why use synthetic inputs?

Answer: They isolate timers, state transitions and counters from camera and landmark errors. Controlled timestamps make expected behavior reproducible. The limitation is that successful synthetic tests do not demonstrate recognition performance on real people.

Q34. What does deterministic mean?

Answer: The same inputs, initial state and controlled times produce reproducible outputs. The tests patch the clock so they do not depend on how fast the computer executes the script.

Q35. Does 20 Hz mean your camera runs at 20 FPS?

Answer: No. It means the test supplies 20 updates per simulated second. Camera FPS is a property of an integrated capture pipeline and must be measured. Test frequency and actual camera throughput are different quantities.

Q36. Why start a fresh engine for each scenario?

Answer: To isolate the tests. Otherwise old yawns, scores and timers could contaminate the next scenario. During one continuous stream we do the opposite: reuse the same engine so its history is preserved.

Q37. Does eight out of eight mean 100% accuracy?

Answer: No. It means eight groups of explicit software assertions passed. Recognition accuracy requires labeled predictions over real samples or events and a defined evaluation protocol. Test-case pass rate is not detection accuracy.

Q38. What do your line graphs prove?

Answer: They show the engine's response to a specified synthetic sequence, including filtering delay, score growth, warning transitions and recovery. They demonstrate software behavior, not discrimination between sleepy and non-sleepy people.

Q39. What does the bar chart show?

Answer: It compares delays at 10, 20 and 30 Hz. Filtered onset delays were 0.30, 0.15 and 0.10 seconds; first DROWSY delays were 1.80, 1.30 and about 1.17 seconds. It reveals rate dependence, not an optimal operating rate.

Q40. Why are the dataset images included if they were not tested?

Answer: They illustrate a published candidate evaluation resource and its labels. The caption identifies them as literature examples. They should not be presented as screenshots of our detections or evidence that the project was evaluated on that dataset.

Q41. What is the use of the literature table?

Answer: It compares previous datasets, methods, reported measurements and gaps. It motivates our design and evaluation choices. Accuracy numbers in that table belong to cited studies and cannot be transferred to our engine.

Q42. Why record a SHA-256 hash?

Answer: It fingerprints the source file associated with the results so a changed implementation can be distinguished. It supports traceability, not proof of correctness or security certification.

E. Difficult questions and failure cases

Q43. Can you tell talking from yawning?

Answer: Only partially. The duration rule rejects some brief openings, but speech, singing or open-mouth breathing can meet the same MAR condition. We have not tested real examples to quantify confusion. Richer temporal visual features and labeled evaluation are needed.

Q44. Can you tell voluntary eye closure from sleepiness?

Answer: No reliable cause distinction is implemented. Both can produce the same low EAR and duration. We detect timing-defined closure evidence; the warning label is not proof of why the person closed their eyes.

Q45. What happens if the person looks away?

Answer: Head pose can change apparent eye geometry. The engine has no gaze, pose or landmark-confidence input. Filtering may reduce brief fluctuations but cannot correct a sustained misleading EAR value.

Q46. What if the person is sleepy with eyes open?

Answer: The engine can miss that condition if its chosen cues do not supply enough evidence. Drowsiness is broader than visible eye closure and yawning. The absence of a warning does not prove alertness.

Q47. Does it work with glasses or in low light?

Answer: That has not been established. Those conditions affect upstream images and landmark quality, and the current tests bypass that stage. We would need representative recordings, quality checks and measured results before claiming robustness.

Q48. What happens when the face is lost?

Answer: There is no explicit missing-input or unavailable-state policy in this engine. If updates stop during a stored CLOSED state, later timing can extend across the gap. A front end should flag invalid observations and define whether to pause, reset or mark the state unavailable.

Q49. Why do update rates affect the warnings?

Answer: The median window is seven samples rather than a fixed duration, and score changes occur per update. Higher rates compress the filter window in seconds and perform more score updates each second. A time-based redesign would need matched evaluation.

Q50. What is wrong with time.time()?

Answer: It reads wall-clock time, which may be adjusted. Elapsed live durations are better measured with a monotonic clock. Recorded videos should use their timestamps so playback or processing speed does not alter event duration. The current code has not yet made those changes.

Q51. Is the system suitable for deployment now?

Answer: The current evidence supports a prototype temporal component, not deployment readiness. The camera front end, missing-input policy, representative real-video performance and end-to-end timing still need integration and evaluation.

F. Evaluation design, implementation and your work

Q52. How would you measure accuracy properly?

Answer: First define the target: blinks, yawns, sustained closures or labeled warning states. Use independently labeled videos and separate people for calibration and testing. Match predicted events to ground truth using a stated tolerance, then report the relevant metrics with uncertainty.

Q53. What are precision, recall and F1?

Answer: Precision is TP/(TP+FP): how many predicted events were correct. Recall is TP/(TP+FN): how many real events were found. F1 = 2PR/(P+R) balances them. TP means true positive, FP false positive and FN false negative; handle zero denominators explicitly in evaluation.

Q54. What are a confusion matrix and macro-F1?

Answer: A confusion matrix counts predicted labels against true labels. Macro-F1 computes F1 separately for each class and averages equally, so a common class does not automatically dominate the result. Both require independently defined reference labels.

Q55. Why separate subjects instead of random frames?

Answer: Frames from the same person or adjacent clip are highly similar. Sharing them between calibration and testing can leak person-specific information. Subject separation better evaluates behavior on people not used to choose thresholds.

Q56. Why measure false warnings per hour?

Answer: A system can have acceptable aggregate classification metrics yet warn too frequently during normal driving behavior. False warnings per hour measures that practical burden. Missed sustained closures and event-onset delay address other important failure modes.

Q57. What is an ablation experiment?

Answer: Remove one component, such as yawning, hysteresis or score smoothing, while holding data and evaluation conditions fixed. Compare results to estimate that component's contribution. We propose these comparisons; the current paper does not report them as completed.

Q58. Which file should I open to explain the algorithm?

Answer: Open temporal_engine.py. Explain __init__ for configuration and state, _clean_old_events for expiry, and update for one complete processing step. Then show verify_engine.py for how the expected behaviors were checked.

Q59. Why use a class and deque?

Answer: A class bundles the state and operations for one stream. A deque stores recent values and supports efficient end operations. The seven-value feature histories are bounded; the event histories are time-expired. The closed-eye ratio still sums retained entries on each update.

Q60. Is every update constant time?

Answer: Not as implemented. The ratio calculation sums the retained closed-eye flags, so its work grows with history length. Other components are small, but that does not justify claiming constant-time or measured real-time performance.

Q61. What would you improve first?

Answer: Define and integrate the image/landmark front end, add confidence and missing-data handling, use appropriate timestamps, and evaluate ambiguity cases with independent labels. Then tune thresholds and compare components on separate calibration and test subjects.

Q62. What is the difference between an intended pipeline and completed work?

Answer: An intended diagram explains how a full application could be assembled. Completed work requires the corresponding code and evidence that it runs and was evaluated. Here the temporal engine and synthetic checks are present; the camera and landmark stages are not implemented in this checkout.

Q63. How should I explain tool-assisted code, figures or writing?

Answer: Describe assistance honestly under your institution's requirements. Explain what you reviewed, changed and verified yourself. Do not claim to have personally authored every line or collected data that you did not collect. Understanding and demonstrating the logic is stronger than inventing a development history.

Q64. What if I do not know an answer or do not have a measurement?

Answer: Say so precisely: That was not measured in this evaluation, or That behavior is not handled explicitly. Then explain the relevant code and a concrete evaluation or improvement. Never replace an unknown with a guessed accuracy, speed or dataset claim.

10. Final revision sheet

A 45-second project explanation

Drishti is a rule-based temporal decision engine for driver-monitoring features. It accepts eye and mouth aspect ratios, applies a seven-observation median filter, uses hysteresis and timers to track events, and combines current closure with recent eye and yawn history. A bounded evidence score and prioritized rules produce ALERT, FATIGUED, DROWSY or CRITICAL. We checked eight controlled synthetic scenarios, all of which passed their assertions, and investigated closure recovery and sampling-rate effects. These results verify selected software behavior. Camera integration, reliable distinction from competing behaviors, and real-driver performance remain to be evaluated.

Numbers worth remembering

Item	Value
Eye close / reopen	EAR <= 0.23 / EAR >= 0.27
Median history	Up to 7 observations per feature
Blink duration	0.08-0.65 seconds, counted on reopening
Long-closure registration	At least 0.8 seconds; once per continuous closure
Yawn start / confirm / end	MAR >= 0.42 / persists >= 0.65 s / MAR <= 0.28
Recent memory	Yawns 45 s; long closures and eye samples 30 s
Direct DROWSY / CRITICAL closure	1.5 s / 3.0 s; other combined branches also exist
Displayed-score smoothing	Rise 0.20; fall 0.08; extra recovery 0.12 per eligible update
Functional verification	8 scenarios at 20 Hz using synthetic features and controlled time

Claims to avoid

- Do not say 100% accuracy because eight tests passed.
- Do not call the score a probability or percentage of sleepiness.
- Do not claim the sample proxy measures actual 80% eyelid coverage.
- Do not say we trained on YawDD or UTA-RLDD.
- Do not claim the code reliably distinguishes talking, breathing or deliberate closure.
- Do not claim camera integration, alarms, low-light robustness or processing speed without evidence.
- Do not invent who implemented each part; explain your own demonstrable contribution.

Glossary for quick recall

Threshold: Numerical boundary. **Hysteresis:** Different entry/exit thresholds with state memory. **Median:** Middle ordered value. **Temporal:** Across time. **Fusion:** Combining evidence. **Proxy:** Indirect approximation. **Window:** Retained recent observations or events. **Calibration:** Selecting parameters using designated data. **Assertion:** Check that an expected condition is true. **Latency:** System processing delay. **Ground truth:** Independent reference labels. **Baseline:** Simpler comparison method. **Ablation:** Component-removal comparison. **Generalization:** Performance beyond the data or people used to select the method.

Source guide: Main factual basis: temporal_engine.py; research_paper/support/verify_engine.py and verification_results.json; research_paper/rewrite_build/make_ieee_figures.py; and the current Drishti_IEEE_Illustrated_Research_Paper.docx. Earlier paper references provide the literature background. These notes do not introduce new empirical results.